arXiv:2407.12828v3 [cs.CL] 20 Jul 2025

Why Does New Knowledge Create Messy Ripple Effects in LLMs?

Jiaxin Qin[1*], **Zixuan Zhang**[1], **Pengfei Yu**[1,3], **Manling Li**[2], **Heng Ji**[1]
[1]University of Illinois Urbana-Champaign
[2]Stanford University
[3]Boson AI
{qjx0814, zixuan11, pengfei4, hengji}@illinois.edu
manlingl@stanford.edu

Abstract

Extensive previous research has focused on post-training knowledge editing (KE) for language models (LMs) to ensure that knowledge remains accurate and up-to-date. One desired property and open question in KE is to let edited LMs correctly handle *ripple effects*, where LM is expected to answer its logically related knowledge accurately. In this paper, we answer the question of why most KE methods still create messy ripple effects. We conduct extensive analysis and identify a salient indicator, *GradSim*, that effectively reveals when and why updated knowledge ripples in LMs. GradSim is computed by the cosine similarity between gradients of the original fact and its related knowledge. We observe a strong positive correlation between ripple effect performance and GradSim across different LMs, KE methods, and evaluation metrics. Further investigations into three counter-intuitive failure cases (*Negation*, *Over-Ripple*, *Multi-Lingual*) of ripple effects demonstrate that these failures are often associated with very low GradSim. This finding validates that GradSim is an effective indicator of when knowledge ripples in LMs. The code is available.[2]

1 Introduction and Related Work

Large language models (LLMs) can serve as powerful knowledge bases (KBs) thanks to their impressive knowledge storage, retrieval, and reasoning capabilities (Petroni et al., 2019; AlKhamissi et al., 2022; Zhang et al., 2024). However, real-world knowledge keeps updating and evolving constantly, which motivates extensive research efforts on post-training knowledge editing (KE) (Meng et al., 2022, 2023; Yin et al., 2023; Zhong et al., 2023a; Song et al., 2024; Liu et al., 2024) to make sure that the knowledge in LMs remains accurate and up-to-date.

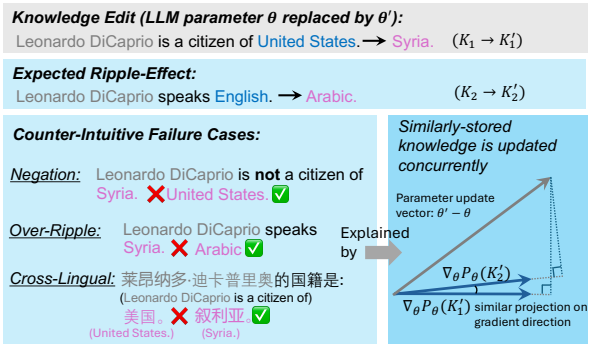


Figure 1: An illustration of ripple effects in LLM knowledge editing. Our work empirically demonstrates the positive correlation between gradient similarity explains a large portion of the ripple effect. Furthermore, messy similarities between knowledge points create several counter-intuitive ripple effect failures.

Previous research has proposed numerous evaluation metrics to ensure the efficiency and consistency of these editing methods. Among them, one critical criterion is the ability of the KE method to handle *ripple effects* (Cohen et al., 2023), where a single edit should automatically and accurately propagate to related facts. For example, suppose an edit changes *Leonardo DiCaprio*'s nationality to *Syrian*. The model should automatically update its related information, such as knowing that his primarily used language is now *Arabic*. Such a task is very challenging because it requires the model to correctly understand and infer complex relationships among knowledge elements and accurately locate their parametric storage in order to perform the edits. Empirically, even though direct knowledge edits typically achieve over $> 90\%$ accuracy, the success rates of ripple effects struggle to exceed 50% across all recent KE methods, even on the simplest task in RippleEdits (Cohen et al., 2023).

In this paper, we answer the intriguing research question of when and why updated knowledge ripples in language models. We hypothesize that the knowledge storage among parameters plays a critical role in determining the ripple effects between

[1]Work done during internship at UIUC.
[1]https://github.com/JiaxinQin0814/Ripple_Effect_Analysis.

knowledge facts. A messy relationship among knowledge elements can make achieving a successful ripple effect intractable or impossible. Intuitively, the similarity of knowledge storage should be an important factor, as knowledge represented by similar parameters will respond similarly to parameter updates during knowledge editing. Following this intuition, we conduct extensive analysis and identify a salient indicator that strongly reveals how likely an updated fact will ripple in a language model: the *cosine similarity* between the *gradients of the related knowledge facts* (GradSim). We use gradients to represent knowledge storage distribution in LMs because they indicate which parameters in the model are responsible for increasing or decreasing the likelihood of answering certain knowledge. We observe a strong positive correlation between ripple effect performance and the cosine similarity of gradients across different LMs, editing methods, and evaluation metrics, with a Pearson correlation metric reaching as high as 0.85.

The hypothesis and analysis above predict a counter-intuitive phenomenon: knowledge with similar parameter-storing locations, even if logically unrelated or contradictory, will create positive ripple effects toward each other, and vise versa. Viewing GradSim as an indicator of ripple effects, we verify this paradox above by discovering and explaining three specific ripple effect cases: ***Negation***, ***Over-Ripple Errors*** and ***Cross-Lingual Transfer***. As illustrated in Figure 1, assuming that a knowledge edit changes the citizenship of *Leonardo DiCaprio* from *United States* to *Syria*. In ***Negation***, LMs with different sizes including GPT2-XL and LLaMA-2 unexpectedly answer the negated query *Leonardo is not a citizen of* still by *Syria* instead of logically correct answers such as *United States*. Moreover, in ***Over-Ripple Errors***, the LMs over-memorize the edit target *Syria*, and tends to always answer *Syria* even when asked about other topics such as language. In ***Cross-Lingual Transfer***, even the most powerful cross-lingual LMs could make mistakes when asked about the edited knowledge in a different language. All of these ripple-effect cases are commonly encountered in real-world applications, but are more challenging and often experience counter-intuitive failures with current LMs and KE methods. In our experiments, we demonstrate that the model's failure in these cases is strongly correlated with a too small GradSim, the similarity in knowledge distributions within LMs.

2 GradSim: A Ripple Effect Indicator

In this section, we formally introduce GradSim, a ripple-effect indicator based on the knowledge storage similarity between related knowledge. We use x and y to denote a pair of original fact and its related knowledge respectively, and we use (q_x, a_x) and (q_y, a_y) to represent query-answer pairs based on the corresponding knowledge facts. For instance, if q_x and a_x are <*Leonardo DiCaprio is a citizen of*> and <*United States*> respectively, then one example pair of q_y and a_y could be <*Leonardo DiCaprio speaks*> and <*English*>. Given a query q_x, typical KE methods update a_x to a new answer a'_x by applying an update on the model parameters θ, and ripple effect evaluations expect that the LM can automatically find the correct a'_y when asked q_y. Based on our hypothesis, knowledge represented by similar parameteres will respond similarly to parameter updates during knowledge editing. We employ the gradient to model the storage of knowledge within an LLM, and use the cosine similarity to measure the proximity between the storage distribution of two pieces of knowledge:

$$\textit{GradSim}(x, y) = \cos(\nabla_\theta P_\theta(a_x|q_x), \nabla_\theta P_\theta(a_y|q_y))$$

3 Experiments

We assess the effectiveness of GradSim by empirically examining its correlation with ripple effect performance, aiming to determine if it reliably indicates successful knowledge propagation in language models. Furthermore, we analyze three typical counter-intuitive failure cases in detail to understand the role of GradSim in these situations.

3.1 Data, Models, and KE Methods

In our experiments, we mainly employ *RippleEdits* (Cohen et al., 2023), the most widely-used benchmark for evaluating ripple effects for knowledge editing methods in LLMs. We mainly use the *popular* split in *RippleEdits* because popular entities are more likely to be recognized by language models. This approach helps to minimize any side effects resulting from the model's lack of knowledge, allowing us to focus on its reasoning abilities. To ensure a comprehensive evaluation, we also demonstrate results in the *recent* split as shown in Figure 2. In our experiments, we consider two typical knowledge editing (KE) methods: ROME (Meng et al., 2022) and MEMIT (Meng et al., 2023), which are

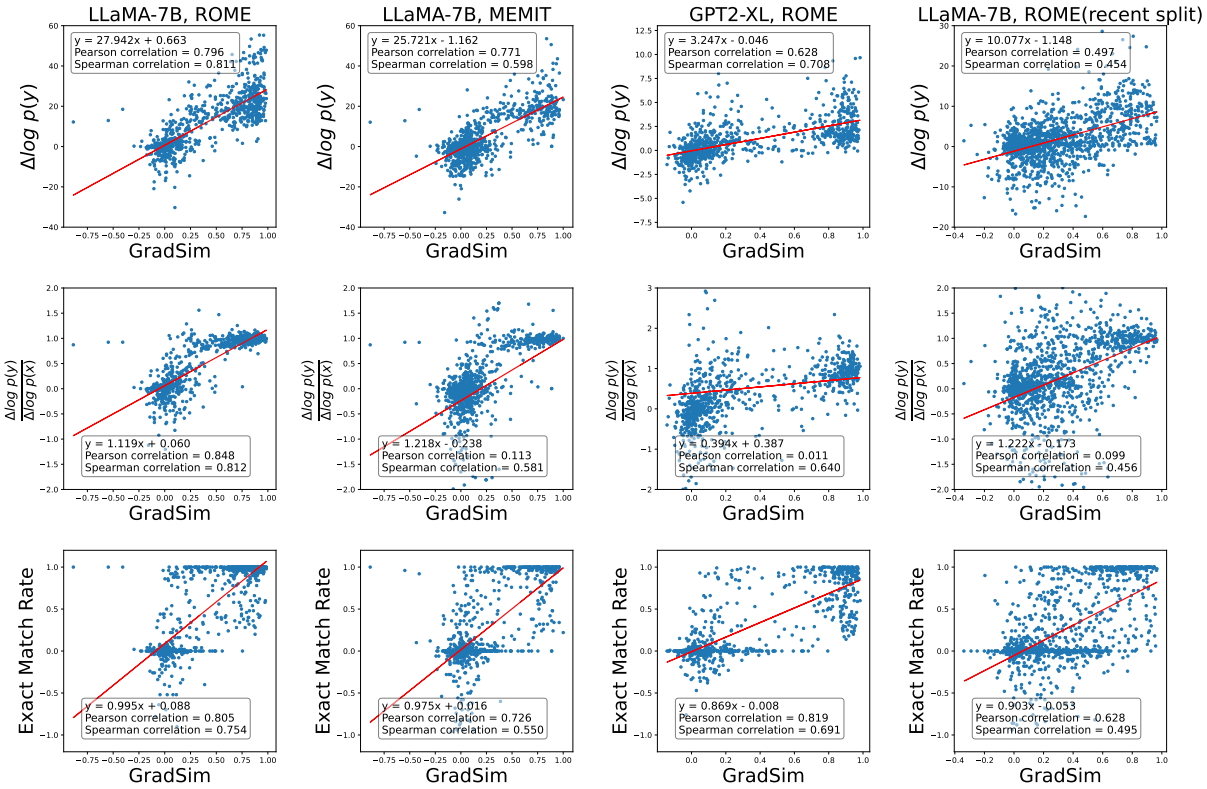


Figure 2: Main results of evaluating the correlation between ripple effect performances and GradSim values.

based on the locate-and-edit approach to modify model parameters. For language models, we evaluate both larger models like LLaMA2-7B (Touvron et al., 2023) and smaller models like GPT-2 XL (Radford et al., 2019) to ensure a comprehensive evaluation and maintain consistency with the original settings in ROME and MEMIT.

3.2 Evaluation Metrics of Ripple Effects

To ensure the validity of our experiment results, we consider multiple evaluation metrics assessing how well the model performs in answering ripple-effect queries. These metrics include both accuracy-based measures, such as the exact match rate, and more quantified likelihood metrics, such as the absolute and relative gains in likelihood.

Exact-Match (EM) Rate Similar to (Cohen et al., 2023), we first consider accuracy-based metrics to calculate the proportion of correct answers the model generates from multiple random sampling choices. For each ripple query q_y, we sample 50 generated answers with a temperature 0.7, and compute the proportion of answers that include the correct answer. Our metric differs slightly from that in (Cohen et al., 2023), as we need to compute performance for each individual data point to analyze the overall correlation. The maximum length for generation is set to a small size of 15, as we believe that the answer is expected to appear early in a cloze-test query format.

Absolute Likelihood Gain We also examine the answer probabilities to obtain a more detailed and quantifiable assessment of the performance. As probability values could be very small as the sequence length increases, we use log-likelihood score $\log P(a'_y|q_y)$, and measure its absolute gain on the correct answers before and after editing:

$$\Delta \log P(y) = \log P_{\theta'}(a'_y \mid q_y) - \log P_{\theta}(a'_y \mid q_y).$$

Relative Likelihood Gain This metric is formulated by dividing the absolute gain of ripple effects by the absolute gain of the original fact, thereby normalizing the difficulty of the knowledge editing itself.

$$\frac{\Delta \log P(y)}{\Delta \log P(x)} = \frac{\log P_{\theta'}(a'_y \mid q_y) - \log P_{\theta}(a'_y \mid q_y)}{\log P_{\theta'}(a'_x \mid q_x) - \log P_{\theta}(a'_x \mid q_x)}.$$

3.3 Main Results

We conduct a comprehensive correlation analysis across different language models, knowledge editing methods, and performance metrics for ripple effects, with the results illustrated in Figure 2. A strong positive correlation is observed between